SERVICIO NACIONAL DE APRENDIZAJE – SENA

CENTRO DE LA INDUSTRIA, LA EMPRESA Y LOS SERVICIOS (CIES)

REGIONAL HUILA

Programa de Formación: Tecnología en Análisis y Desarrollo de Software (ADSO)

Ficha: 3145556

MANUAL DE INSTALACIÓN

Proyecto: Monitor de Consumo Eléctrico

Procedimiento para desplegar el API gateway y los servicios, la cola de mensajes, el broker MQTT, el dispositivo de medición, la aplicación web y la aplicación móvil

Dato	Información
Integrantes	Juan Esteban Cortés Parra, Miguel Ángel García Artunduaga
Instructor	Equipo de instructores de la ficha 3145556
Centro de Formación	Centro de la Industria, la Empresa y los Servicios (CIES) – SENA Regional Huila
Programa	Tecnología en Análisis y Desarrollo de Software (ADSO)
Ficha	3145556
Tipo de documento	Manual de instalación
Versión	1.4
Fecha de elaboración	07 de octubre de 2026 (actualización: 28 de octubre de 2026)

Año 2026

2. Control de versiones del documento

El historial de cambios del presente manual se registra en la siguiente tabla. Cada modificación del proceso de instalación que afecte el software debe verse reflejada en una nueva versión del documento.

Versión	Fecha	Descripción del cambio	Responsable
1.0	07/10/2026	Creación inicial del manual de instalación para backend, frontend web y frontend móvil, con requisitos de la especificación de requisitos y referencias en formato APA.	Equipo desarrollador
1.1	13/10/2026	Actualización al estado final: cinco piezas del sistema, variables de entorno completas, broker MQTT, firmware, vinculación del primer dispositivo, producción, aplicación móvil integrada con la API y verificaciones y soluciones ampliadas.	Equipo desarrollador
1.2	24/10/2026	Arquitectura de servicios: clonado de todos los repositorios (incluido `en-monitor-measurement-service`), infraestructura en `energy-monitor-infra` (pila completa y modo desarrollo con MySQL, RabbitMQ y Mosquitto), puertos 8080, 8082, 8083, 8084 y 8085, variables por servicio (`MQTT_DEVICE_SERVICE_*`, `MQTT_MEASUREMENT_SERVICE_*` en lugar de `MQTT_BACKEND_*`, `INTERNAL_API_TOKEN`, `RABBITMQ_*`, `MEASUREMENT_CONTEXT`), broker autenticado por el servicio de dispositivos, firmware en `energy-monitor-backend/firmware/`, verificaciones y soluciones actualizadas.	Equipo desarrollador
1.3	27/10/2026	Migración terminada: siete microservicios (seguridad 8082, dispositivos 8084, mediciones 8085, hogares 8086, notificaciones 8087, alertas 8088 y recomendaciones 8089) detrás del gateway y backend retirado; clonado, `.env` por servicio, orden de arranque, interruptores de consumidores y relays, primer administrador con `make-admin.sh`, nuevas pruebas de punta a punta, verificaciones y soluciones actualizadas.	Equipo desarrollador
1.4	28/10/2026	Alerta de consumo inusual en el servicio de alertas (sin variables ni pasos de instalación nuevos), fotos de perfil y de miembros, y centro de ayuda de Ajustes; la versión final se unifica en 1.4.	Equipo desarrollador

3. Tabla de contenido

2. Control de versiones del documento

3. Tabla de contenido

4. Introducción

5. Objetivo

6. Alcance técnico

7. Requisitos mínimos

- 7.1 Requisitos del equipo (según la especificación de requisitos)
- 7.2 Recursos de hardware recomendados para ejecutar el sistema
- 7.3 Requisitos de software para la instalación

8. Variables de entorno que el instalador debe conocer

9. Instalación de los servicios del backend

- 9.1 Obtener el código · 9.2 Definir las variables de entorno (.env) · 9.3 Levantar la infraestructura con Docker Compose · 9.4 Crear los archivos locales de cada servicio · 9.5 Generar el par de claves RSA (tokens JWT) · 9.6 Generar el pepper de recuperación · 9.7 Decidir el correo de recuperación · 9.8 Compilar y arrancar

10. Instalación del frontend web

- 10.1 Dependencias · 10.2 Configurar el gateway de desarrollo · 10.3 Arrancar en desarrollo · 10.4 Permiso de CORS en el gateway

11. Instalación de la aplicación móvil

- 11.1 Dependencias · 11.2 Configurar la URL de la API · 11.3 Arrancar Expo · 11.4 Probar en un dispositivo físico · 11.5 Notas de compatibilidad

12. Broker MQTT

13. Firmware del dispositivo de medición

14. Vincular el primer dispositivo

15. Producción

16. Correo local con Mailpit (opcional)

17. Verificación de la instalación

18. Resumen de comandos de instalación

19. Solución de problemas habituales

20. Referencias (formato APA, 7.ª edición)

4. Introducción

El presente documento es el manual de instalación del software «Monitor de Consumo Eléctrico». Su propósito es que cualquier desarrollador, instructor o técnico —sin importar su experiencia previa con el proyecto— sea capaz de obtener el código fuente, configurar las dependencias, echar a andar los componentes del sistema y verificar que funcionen de forma conjunta en un equipo de trabajo.

El sistema se compone de estas piezas, cada una en su repositorio: un API gateway, única entrada de la API (Spring Cloud Gateway sobre Spring Boot 4.0.8), siete microservicios —seguridad, dispositivos, mediciones, hogares, notificaciones, alertas y recomendaciones—, todos construidos con Java 21 y Spring Boot 4.1.1; una aplicación web construida con React 19 y Vite 8; una aplicación móvil Android construida con Expo SDK 57 y React Native 0.86; un broker MQTT Eclipse Mosquitto que autentica a cada cliente contra el servicio de dispositivos; y un dispositivo de medición formado por una placa ESP32 y un sensor PZEM-004T V3 (Spring, s. f.; Vite, s. f.; Expo, s. f.; Eclipse Foundation, s. f.). El antiguo backend modular (`energy-monitor-backend`) está retirado y ya no se instala: solo conserva el firmware del dispositivo. La base de datos MySQL 8.4, la cola de

mensajes RabbitMQ 4.1 y el broker se levantan con Docker Compose desde el repositorio de infraestructura, y la entrega del correo puede simularse localmente con Mailpit (Docker Inc., s. f.; RabbitMQ, s. f.; Mailpit, s. f.).

La información consignada fue obtenida directamente de los repositorios del proyecto (gateway, servicios, web, móvil e infraestructura), de la especificación de requisitos y de los README incluidos en cada repositorio, por lo que corresponde a la rama `dev` de los repositorios el 27 de octubre de 2026. Las referencias se formatearon según el estilo APA, séptima edición (American Psychological Association [APA], 2020).

5. Objetivo

Permitir que un equipo de desarrollo nuevo o de mantenimiento ponga en marcha el sistema «Monitor de Consumo Eléctrico» en un computador Windows, macOS o Linux, a partir de los repositorios oficiales, sin depender de documentación informal ni de conversaciones pasadas.

- Enumerar los requisitos mínimos de hardware, sistema operativo y software, en coherencia con la especificación de requisitos.
- Establecer versiones exactas de las herramientas que el proceso de instalación requiere (Java, Node, Docker, OpenSSL, Arduino).
- Describir, paso a paso y con los comandos concretos, la instalación de la base de datos, la cola de mensajes, los servicios del backend, la aplicación web, la móvil, el broker MQTT y el firmware del dispositivo.
- Presentar las variables de configuración que el instalador debe definir antes del primer arranque.
- Indicar cómo vincular el primer dispositivo, cómo desplegar en producción, cómo verificar que cada componente quedó operativo y cómo diagnosticar los errores más habituales.

6. Alcance técnico

El manual cubre la instalación en un equipo de desarrollo local de los siguientes componentes y, en la sección 15, el resumen del despliegue en un servidor de producción con Docker Compose:

- MySQL 8.4, RabbitMQ 4.1 y el broker MQTT Mosquitto mediante Docker Compose (energy-monitor-infra), en dos modos: la pila completa en contenedores o solo la infraestructura, con los servicios ejecutados en el equipo (modo desarrollo).
- API gateway (en-monitor-api-gateway, puerto 8080).
- Servicios de seguridad (en-monitor-security-service, 8082), dispositivos (en-monitor-device-service, 8084), mediciones (en-monitor-measurement-service, 8085), hogares (en-monitor-home-service, 8086), notificaciones (en-monitor-notification-service, 8087), alertas (en-monitor-alert-service, 8088) y recomendaciones (en-monitor-recommendation-service, 8089), con perfil local y pruebas JUnit.
- Firmware del dispositivo, en la carpeta `firmware/` del repositorio retirado `energy-monitor-backend` (solo se clona para el firmware; el backend ya no se ejecuta).
- Aplicación web (Energy-Monitor-application) con Vite.
- Aplicación móvil Android (Energy-Monitor-Mobile) con Expo y Expo Go.
- Utilidades: `make-admin.sh` para el primer administrador; opcionales, Mailpit para correo local, claves RSA propias para los JWT y las pruebas de punta a punta de `energy-monitor-infra/e2e/`.

El repositorio `energy-monitor-devices` está retirado: el firmware pasó a `energy-monitor-backend/firmware/` y la configuración del broker a `energy-monitor-infra/mosquitto/`. No cubre la contratación de un servidor ni de un dominio, ni la configuración detallada de un proxy inverso; el procedimiento completo de producción está en el manual de implementación y despliegue.

7. Requisitos mínimos

7.1 Requisitos del equipo (según la especificación de requisitos)

De acuerdo con los requisitos de compatibilidad de la especificación de requisitos (RNF-COM), el software se ejecuta en equipos de cómputo y dispositivos móviles en perfecto estado, con conexión a internet, almacenamiento suficiente para guardar la aplicación y la última versión disponible del sistema operativo (Cortés Parra y García Artunduaga, 2026):

- Sistema operativo de computador: Windows 10 o superior.
- Sistema operativo móvil: la última versión disponible de Android. La aplicación móvil no está disponible para iPhone.
- Navegador web actualizado: Chrome, Firefox, Safari o Edge. La vinculación del dispositivo por Bluetooth solo está disponible en Chrome o Edge de escritorio y en Chrome para Android, y exige una conexión segura (HTTPS o `localhost`).
- Conexión a internet (los servicios se comunican mediante protocolos estándar HTTP/HTTPS).
- Para el dispositivo de medición: una placa ESP32 DevKit con Wi-Fi y Bluetooth, un sensor PZEM-004T V3, una fuente HLK-PM03 y un cable USB.

7.2 Recursos de hardware recomendados para ejecutar el sistema

Más allá de los requisitos del equipo indicados en la especificación de requisitos, el equipo donde se instalen los componentes debe contar con los siguientes recursos mínimos para una experiencia sin cuelgues ni errores de memoria. Las versiones exactas de software se detallan en la tabla de la sección 7.3.

Recurso	Mínimo	Recomendado
Sistema operativo (PC)	Windows 10 de 64 bits (con Docker Desktop y WSL 2)	Windows 11 de 64 bits o Linux (Ubuntu 24.04+)
Sistema operativo (móvil)	Android 10 o superior	Última versión de Android disponible
Procesador	Intel Core i3 o equivalente	Intel Core i5 o equivalente
Memoria RAM	8 GB	16 GB (Docker Desktop, MySQL, RabbitMQ, el gateway y los siete servicios Java, Node y herramientas de diseño se abren con holgura)
Espacio en disco libre	8 GB (código, node_modules, imagen MySQL y contenedores)	20 GB o más (para varias versiones, cachés de Docker y archivos de la base de datos)
Conectividad	Internet estable (para clonar, descargar paquetes y Expo Go)	Conexión de banda ancha
Almacenamiento adicional	Al menos 5 GB libres para datos de prueba y respaldos de MySQL	Espacio a libre disposición

> *Si el equipo cumple el mínimo pero no la recomendación, instala solo los componentes que necesites de forma simultánea (p. ej. usa la pila completa en contenedores en lugar de abrir los ocho servicios Java en el IDE, y no abras Android Studio a la vez).*

7.3 Requisitos de software para la instalación

Las versiones mínimas verificadas sobre los repositorios de los servicios, de los dos frontends y de infraestructura son:

Herramienta	Versión mínima	Para qué se usa
Java JDK	21 o superior	Compilar y ejecutar el gateway y los servicios (Spring Boot 4.1.1; el gateway, 4.0.8).
Node.js	22 LTS (mín. 20.19)	Tooling de React 19/Vite 8 y de Expo SDK 57.
npm	incluido con Node	Gestión de dependencias del web y del móvil.
Docker Desktop / Docker Engine	24 o superior	Levantar MySQL 8.4, RabbitMQ 4.1, el broker Mosquitto, la pila completa y Mailpit.
Docker Compose	2.24 o superior	Crear el esquema y el usuario de base de datos y levantar la infraestructura; el modo desarrollo usa la etiqueta `!override`.
OpenSSL y bash	cualquier versión reciente	Generar el par de claves RSA, el pepper, los secretos y los certificados de desarrollo (`prepare.sh`).
Git	última estable	Clonar los repositorios y, en Windows, trae OpenSSL embebido.
Expo Go	app de la tienda	Probar la app móvil sin emulador en un dispositivo físico.
arduino-cli o Arduino IDE 2	última estable	Compilar y cargar el firmware del dispositivo (núcleo esp32 de Espressif 3.x).

Todas estas herramientas se distribuyen con documentación oficial en línea (Oracle, s. f.; Docker Inc., s. f.; Node.js Foundation, s. f.; Git, s. f.; OpenSSL, s. f.; Expo, s. f.; Arduino, s. f.).

8. Variables de entorno que el instalador debe conocer

Ningún servicio guarda credenciales en el código: todo se inyecta por variables de entorno o por archivos ignorados por Git (`.env`, `application-local.yaml`). Hay dos niveles de configuración:

- `infra/.env` (copia de `energy-monitor-infra/.env.example`): lo lee Docker Compose y sirve para la pila completa y para la infraestructura del modo desarrollo.
- **El `.env` de cada servicio** (copia de su `.env.example`) y su `application-local.yaml`: solo en modo desarrollo, cuando el servicio corre en el equipo. Los valores compartidos deben ser los mismos que en `infra/.env`; `MYSQL_PASSWORD` se llama allí `SPRING_DATASOURCE_PASSWORD`.

Los nombres que siguen son los exactos de los `.env.example` y de los `application.yaml` de cada repositorio (Spring, s. f.).

> *Los archivos `.env` se leen como `.properties`: escribe los valores sin comillas, sin espacios al final y sin `$`. Un valor con `$` se interpreta como un marcador y Compose también lo interpreta. No dejes los textos de ejemplo `CHANGE_ME_...`.*

Variable / archivo	Para qué sirve	Dónde anotarla
MYSQL_PASSWORD / MYSQL_ROOT_PASSWORD	Contraseñas del usuario `energy_app` de los servicios y de `root`. MySQL las toma solo al crear su volumen.	`infra/.env`
SPRING_DATASOURCE_PASSWORD	Misma contraseña que MYSQL_PASSWORD; la usan cada servicio en modo desarrollo y sus pruebas del perfil `test`.	`.env` de cada servicio con base de datos
SECURITY_RESET_PEPPER	Pepper largo y aleatorio (HMAC) para el código de recuperación.	`infra/.env` y `.env` del servicio de seguridad
RABBITMQ_USERNAME / RABBITMQ_PASSWORD	Usuario de la cola de mensajes (`energy-monitor` por defecto). RabbitMQ lo crea solo al iniciar su volumen.	`infra/.env` y `.env` de cada servicio

Variable / archivo	Para qué sirve	Dónde anotarla
INTERNAL_API_TOKEN	Token de las llamadas `/internal/**` entre servicios; el mismo valor en todos.	`infra/.env` y `.env` de cada servicio
SECURITY_CORS_ALLOWED_ORIGINS	Orígenes permitidos para el navegador; el CORS lo responde el gateway (`http://localhost` en la pila completa, `http://localhost:5173` en modo desarrollo).	`infra/.env` y `.env` del gateway
SECURITY_JWT_PRIVATE_KEY_PATH / SECURITY_JWT_PUBLIC_KEY_PATH	Rutas absolutas a las claves PEM del token JWT. La privada solo la usa el servicio de seguridad; la pública, el gateway y los demás servicios.	`.env` de cada servicio (modo desarrollo)
SECURITY_SESSION_COOKIE_SECURE	`false` en modo desarrollo: la sesión web viaja en cookies HttpOnly y el equipo local se sirve por `http://`.	`.env` del servicio de seguridad
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	Credenciales OAuth («Aplicación web») para iniciar sesión con Google. Si están vacías, no aparece el botón de Google.	`infra/.env` y `.env` del servicio de seguridad
MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM	SMTP para los códigos de verificación, recuperación y borrado y los correos de alertas, recomendaciones y actividad. Solo el servicio de notificaciones envía correo.	`infra/.env` y `.env` del servicio de notificaciones
MQTT_BROKER_URL	Dirección del listener interno del broker (`tcp://localhost:1883` en modo desarrollo).	`.env` de dispositivos y de mediciones
MQTT_DEVICE_SERVICE_USER / MQTT_DEVICE_SERVICE_PASS	Credenciales del servicio de dispositivos en el broker (`em-device-service`). Clave aleatoria (`openssl rand -hex 24`).	`infra/.env` y `.env` de dispositivos
MQTT_MEASUREMENT_SERVICE_USER / MQTT_MEASUREMENT_SERVICE_PASS	Credenciales del servicio de mediciones en el broker (`em-measurement-service`). Las comprueba el servicio de dispositivos, así que deben coincidir en los dos.	`infra/.env` y `.env` de mediciones y de dispositivos
MEASUREMENT_MQTT_ENABLED	`true` para que el servicio de mediciones se suscriba a la telemetría en modo desarrollo.	`.env` del servicio de mediciones
MQTT_CLIENT_ID	Identificador en el broker; debe ser distinto en cada instancia de los servicios de dispositivos o de mediciones.	Variable de entorno (opcional)
MQTT_AUTH_ALLOWED_NETWORKS	Redes desde las que se puede llamar a `/internal/mqtt/auth/**` del servicio de dispositivos (por defecto `127.0.0.1/32,::1/128,172.16.0.0/12`).	Variable de entorno (opcional)
DEVICE_MQTT_PUBLIC_HOST / DEVICE_MQTT_PUBLIC_PORT	Dirección del broker que la web escribe en el dispositivo al vincularlo. Nunca `localhost`: en desarrollo, la IP del PC en la red local; en producción, el dominio del broker. Puerto `8883`.	`infra/.env` y `.env` de dispositivos
DEVICE_OFFLINE_AFTER	Tiempo de silencio tras el cual un dispositivo se marca sin conexión (por defecto `PT150S`); lo usa el servicio de dispositivos.	Variable de entorno (opcional)

Variable / archivo	Para qué sirve	Dónde anotarla
SECURITY_SERVICE_URL, DEVICE_SERVICE_URL, MEASUREMENT_SERVICE_URL, HOME_SERVICE_URL, NOTIFICATION_SERVICE_URL, ALERT_SERVICE_URL, RECOMMENDATION_SERVICE_URL	Direcciones entre servicios en modo desarrollo (`http://localhost:8082`, `:8084`, `:8085`, `:8086`, `:8087`, `:8088` y `:8089`). El gateway usa todas; cada servicio, las de los que llama. Los `.env.example` ya traen los valores locales.	`.env` de cada servicio
NOTIFICATION_WEB_URL	Dirección de la web que usan los enlaces de los correos de alerta (`http://localhost` en la pila completa, `http://localhost:5173` en modo desarrollo).	`infra/.env` y `.env` del servicio de notificaciones
NOTIFICATION_VAPID_PUBLIC_KEY / NOTIFICATION_VAPID_PRIVATE_KEY	Par de claves VAPID de las notificaciones push del navegador. Sin ellas se desactiva el push y el correo sigue funcionando. Deben ser siempre las mismas en un entorno.	`infra/.env` y `.env` del servicio de notificaciones
NOTIFICATION_VAPID_SUBJECT	Contacto asociado a las claves VAPID (por defecto un `mailto:` de soporte).	Variable de entorno (opcional)
EXPO_ACCESS_TOKEN	Token de expo.dev para el push de la aplicación móvil. Va solo en el servidor, nunca en la app.	`infra/.env` y `.env` del servicio de notificaciones
ALERT_LIMIT_ZONE	Zona horaria en la que empiezan el día y el mes de las alertas de límite (por defecto `America/Bogota`).	Variable de entorno del servicio de alertas (opcional)
RECOMMENDATION_EVALUATE_INTERVAL / RECOMMENDATION_ZONE / RECOMMENDATION_INITIAL_DELAY	Cada cuánto se evalúan las reglas de ahorro (`PT1H`), la zona horaria de las reglas por franja del día (`America/Bogota`) y la espera antes de la primera evaluación (`PT2M`).	Variables de entorno del servicio de recomendaciones (opcionales)
SECURITY_EVENTS_CONSUMER_ENABLED, ALERT_EVENTS_CONSUMER_ENABLED, ACTIVITY_CONSUMER_ENABLED, HOME_OUTBOX_RELAY_ENABLED, ACCOUNT_DELETED_CONSUMER_ENABLED, ALERT_CONSUMERS_ENABLED, ALERT_SERVICE_OUTBOX_RELAY_ENABLED, RECOMMENDATION_SCHEDULER_ENABLED, RECOMMENDATION_OUTBOX_RELAY_ENABLED	Interruptores de los consumidores y relays de notificaciones, hogares, alertas y recomendaciones. Todos `true` por defecto; `false` solo servía para convivir con el backend retirado.	`infra/.env` (opcionales; no los definas)
SECURITY_ACCOUNT_DELETION_CODE_MAIL_ENABLED	Envío del código de borrado a las cuentas de Google (por defecto `false`). Actívalo solo con el servicio de notificaciones desplegado.	`infra/.env` y `.env` del servicio de seguridad (opcional)
SECURITY_CONTEXT, DEVICE_CONTEXT, MEASUREMENT_CONTEXT, HOME_CONTEXT, NOTIFICATION_CONTEXT, ALERT_CONTEXT, RECOMMENDATION_CONTEXT, GATEWAY_CONTEXT, FRONTEND_CONTEXT	Rutas de los clones; solo si los repositorios no están uno al lado del otro (sección 9.1).	`infra/.env`
VITE_DEV_API_TARGET	Gateway al que el servidor de Vite reenvía `/api` (por defecto `http://localhost:8080`).	`.env.local` en Energy-Monitor-application (opcional)

Variable / archivo	Para qué sirve	Dónde anotarla
VITE_GOOGLE_CLIENT_ID	El mismo ID de cliente de GOOGLE_CLIENT_ID. Vacío: el botón de Google no se muestra.	`infra/.env` (pila completa) y `.env.local` en Energy-Monitor-application
EXPO_PUBLIC_API_BASE_URL	URL de la API para la app móvil, sin `/api/v1` (sección 11.2).	`.env` en Energy-Monitor-Mobile
EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID	ID de cliente web de Google (el mismo de GOOGLE_CLIENT_ID) para el inicio de sesión nativo de la app.	`.env` en Energy-Monitor-Mobile (opcional)
EXPO_PUBLIC_EAS_PROJECT_ID	Identificador del proyecto de Expo; sin él la app no registra el teléfono para recibir push.	`.env` en Energy-Monitor-Mobile

> *Variables retiradas con el backend:* `BACKEND_URL`, `BACKEND_INTERNAL_URL`, `BACKEND_CONTEXT`, `MQTT_BACKEND_USER` *y* `MQTT_BACKEND_PASS`. *Si siguen en un* `.env` *antiguo, sobran y pueden borrarse.*

9. Instalación de los servicios del backend

Repositorios: `en-monitor-api-gateway` (puerto 8080), `en-monitor-security-service` (8082), `en-monitor-device-service` (8084), `en-monitor-measurement-service` (8085), `en-monitor-home-service` (8086), `en-monitor-notification-service` (8087), `en-monitor-alert-service` (8088) y `en-monitor-recommendation-service` (8089), todos con Java 21 y Spring Boot (4.1.1; el gateway, 4.0.8), MySQL 8.4, Liquibase y JWT RS256, más `energy-monitor-infra` con Docker Compose. La API se consume siempre a través del gateway, en `http://localhost:8080` en modo desarrollo o `http://localhost` con la pila completa (Spring, s. f.).

9.1 Obtener el código

1. Clonar los repositorios uno al lado del otro, con esta estructura de carpetas (infra los construye desde ahí):

```
mkdir energy-monitor && cd energy-monitor
git clone https://github.com/Juan-Cortes2045/energy-monitor-infra.git infra
# Retirado: solo se clona porque guarda el firmware (prepare.sh le escribe dev_ca.h).
git clone https://github.com/Juan-Cortes2045/energy-monitor-backend.git backend/energy-monitor-backend
git clone https://github.com/Juan-Cortes2045/en-monitor-security-service.git backend/en-monitor-security-service
git clone https://github.com/Juan-Cortes2045/en-monitor-device-service.git backend/en-monitor-device-service
git clone https://github.com/Juan-Cortes2045/en-monitor-measurement-service.git backend/en-monitor-measurement-service
git clone https://github.com/angelgarcia0/en-monitor-home-service.git backend/en-monitor-home-service
git clone https://github.com/angelgarcia0/en-monitor-notification-service.git backend/en-monitor-notification-service
git clone https://github.com/angelgarcia0/en-monitor-alert-service.git backend/en-monitor-alert-service
git clone https://github.com/angelgarcia0/en-monitor-recommendation-service.git backend/en-monitor-recommendation-service
git clone https://github.com/Juan-Cortes2045/en-monitor-api-gateway.git backend/en-monitor-api-gateway
git clone https://github.com/Juan-Cortes2045/Energy-Monitor-application.git frontend/energy-monitor-web
git clone https://github.com/angelgarcia0/Energy-Monitor-Mobile.git frontend/energy-monitor-mobile
```

2. Verifica que cada servicio contiene `mvnw`, `pom.xml`, `Dockerfile` y la carpeta `src/`, y que `infra/` contiene `docker-compose.yml`, `docker-compose.dev.yml` y `prepare.sh`. Si algún repositorio queda en otra ruta, indícala en `infra/.env` con su variable `*_CONTEXT` (sección 8).

9.2 Definir las variables de entorno (.env)

Copia el archivo de ejemplo de infraestructura y protégelo. El archivo `.env` está en `.gitignore` y no debe subirse al repositorio:

```
cd infra
cp .env.example .env && chmod 600 .env
# En Windows PowerShell:
Copy-Item .env.example .env
```

Genera los secretos aleatorios que estén vacíos (el bloque no imprime ninguno):

```
gen() { grep -q "^$1=$" .env && sed -i "s|^$1=$|$1=$(openssl rand -hex "$2")|" .env; }
gen MYSQL_PASSWORD 16;    gen MYSQL_ROOT_PASSWORD 16; gen SECURITY_RESET_PEPPER 32
gen RABBITMQ_PASSWORD 24; gen INTERNAL_API_TOKEN 32;  gen MQTT_MEASUREMENT_SERVICE_PASS 24
gen MQTT_DEVICE_SERVICE_PASS 24
```

Pon la IP de LAN del equipo en `DEVICE_MQTT_PUBLIC_HOST` (en Linux, `hostname -I`) y completa, si se usan, el correo, Google, VAPID y Expo (sección 8). Comprueba que no falte ninguna variable obligatoria ni quede un valor de ejemplo:

```
docker compose config -q && echo "compose OK"         # si falta una obligatoria, la nombra
grep -lE '^[A-Z_]+=.*CHANGE_ME' .env ../backend/*/.env  # no debe listar ningún archivo
```

9.3 Levantar la infraestructura con Docker Compose

Primero, desde `infra/`, genera el par RSA de los JWT (`infra/keys/`) y los certificados de desarrollo del broker. Repítelo cada vez que cambie la IP de LAN:

```
./prepare.sh
```

Después elige un modo:

- **Pila completa (todo en contenedores).** Es la forma más rápida de tener el sistema funcionando; no hace falta lo de las secciones 9.4 a 9.8:

```
docker compose up -d --build
docker compose ps        # gateway, security, device, measurement, home, notification, alert y
recommendation deben quedar (healthy)
```

La web queda en `http://localhost` y la API en `http://localhost/api/v1/...`, a través del gateway. Los servicios no publican puertos en este modo.

- **Modo desarrollo (servicios en el equipo).** Levanta solo MySQL, RabbitMQ y el broker, y luego cada servicio desde el IDE o la terminal (secciones 9.4 a 9.8):

```
docker compose stop gateway security device measurement home notification alert recommendation
frontend   # si la pila completa estaba arriba
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d mysql rabbitmq mosquitto
docker compose ps
```

Espera a que `energy-monitor-mysql` y `energy-monitor-rabbitmq` reporten «healthy». Docker crea el esquema `energy_monitor` y el usuario `energy_app` con los valores de `.env`; los dos modos usan la misma base, la misma cola y el mismo broker (Docker Inc., s. f.). La consola de RabbitMQ queda en `http://localhost:15672`.

9.4 Crear los archivos locales de cada servicio

Solo en modo desarrollo. En cada servicio, copia su `.env.example` como `.env` y, si existe, su `src/main/resources/application-local.yaml.example` como `application-local.yaml`:

```
cd ../backend/en-monitor-security-service
cp .env.example .env
cp src/main/resources/application-local.yaml.example src/main/resources/application-local.yaml
# Igual en los servicios de dispositivos, mediciones, hogares, notificaciones, alertas y
recomendaciones.
# En en-monitor-api-gateway solo hay .env.example.
```

Completa cada `.env` con los valores de `infra/.env`:

Archivo	Valores de `infra/.env`	Propios del modo desarrollo
`backend/en-monitor-security-service/.env`	`MYSQL_PASSWORD` (como `SPRING_DATASOURCE_PASSWORD`), `SECURITY_RESET_PEPPER`, `GOOGLE_CLIENT_ID/SECRET`, `RABBITMQ_*`, `INTERNAL_API_TOKEN` (y `HOME_SERVICE_URL=http://localhost:8086`)	`SECURITY_JWT_PRIVATE_KEY_PATH`, `SECURITY_JWT_PUBLIC_KEY_PATH`, `SECURITY_SESSION_COOKIE_SECURE=false`
`backend/en-monitor-device-service/.env`	`MYSQL_PASSWORD`, `RABBITMQ_*`, `INTERNAL_API_TOKEN`, `MQTT_MEASUREMENT_SERVICE_USER/PASS`, `MQTT_DEVICE_SERVICE_USER/PASS`, `DEVICE_MQTT_PUBLIC_HOST`	`SECURITY_JWT_PUBLIC_KEY_PATH`
`backend/en-monitor-measurement-service/.env`	`MYSQL_PASSWORD`, `RABBITMQ_*`, `INTERNAL_API_TOKEN`, `MQTT_MEASUREMENT_SERVICE_USER/PASS`	`SECURITY_JWT_PUBLIC_KEY_PATH`, `MEASUREMENT_MQTT_ENABLED=true`
`backend/en-monitor-home-service/.env`	`MYSQL_PASSWORD`, `RABBITMQ_*`, `INTERNAL_API_TOKEN`	`SECURITY_JWT_PUBLIC_KEY_PATH`, `SECURITY_SERVICE_URL=http://localhost:8082`
`backend/en-monitor-notification-service/.env`	`MYSQL_PASSWORD`, `MAIL_*`, `RABBITMQ_*`, `INTERNAL_API_TOKEN`, VAPID, `EXPO_ACCESS_TOKEN`	`SECURITY_JWT_PUBLIC_KEY_PATH`, `NOTIFICATION_WEB_URL=http://localhost:5173`, `SECURITY_SERVICE_URL`, `HOME_SERVICE_URL`, `DEVICE_SERVICE_URL` (de `localhost`)
`backend/en-monitor-alert-service/.env` y `backend/en-monitor-recommendation-service/.env`	`MYSQL_PASSWORD`, `RABBITMQ_*`, `INTERNAL_API_TOKEN`	`SECURITY_JWT_PUBLIC_KEY_PATH`, `HOME_SERVICE_URL`, `DEVICE_SERVICE_URL`, `MEASUREMENT_SERVICE_URL` (de `localhost`)
`backend/en-monitor-api-gateway/.env`	—	`SECURITY_JWT_PUBLIC_KEY_PATH`, `SECURITY_CORS_ALLOWED_ORIGINS=http://localhost:5173`

En los `application-local.yaml`, la contraseña de `spring.datasource.password` debe ser la misma de `MYSQL_PASSWORD`. Para comprobar que un valor compartido es igual en todos los archivos sin mostrarlo, compara sus resúmenes desde `infra/`:

```
for v in RABBITMQ_PASSWORD INTERNAL_API_TOKEN MQTT_MEASUREMENT_SERVICE_PASS; do
  for f in .env ../backend/*/.env; do
    grep -q "^$v=" "$f" && echo "$v $(grep "^$v=" "$f" | cut -d= -f2- | sha256sum | cut -c1-8)
$f"
  done
done      # cada variable debe dar el mismo resumen en todos sus archivos
```

9.5 Generar el par de claves RSA (tokens JWT)

Los tokens de acceso se firman con RS256. En la pila completa el par lo genera `prepare.sh` en `infra/keys/`, y los servicios del modo desarrollo pueden apuntar a esas mismas claves. También pueden usar un par propio. En Linux y macOS:

```
mkdir -p ~/.energy-monitor/keys
chmod 700 ~/.energy-monitor/keys
openssl genpkey -algorithm RSA -out ~/.energy-monitor/keys/private.pem -pkeyopt
rsa_keygen_bits:2048
openssl rsa -in ~/.energy-monitor/keys/private.pem -pubout -out ~/.energy-monitor/keys/public.pem
```

En Windows (PowerShell o Git Bash), con la misma carpeta oculta `.energy-monitor`:

```
New-Item -ItemType Directory -Force "$HOME\.energy-monitor\keys"
openssl genpkey -algorithm RSA -out "$HOME\.energy-monitor\keys\private.pem" -pkeyopt
rsa_keygen_bits:2048
openssl rsa -in "$HOME\.energy-monitor\keys\private.pem" -pubout -out "$HOME\.energy-
monitor\keys\public.pem"
```

Indica las rutas absolutas (Java no expande `~`) en el `.env` de cada servicio: `SECURITY_JWT_PRIVATE_KEY_PATH` y `SECURITY_JWT_PUBLIC_KEY_PATH` en el servicio de seguridad, y solo `SECURITY_JWT_PUBLIC_KEY_PATH` en el gateway y los demás servicios. Todos deben usar el mismo par.

```
SECURITY_JWT_PRIVATE_KEY_PATH=C:\Users\tu-usuario\.energy-monitor\keys\private.pem
SECURITY_JWT_PUBLIC_KEY_PATH=C:\Users\tu-usuario\.energy-monitor\keys\public.pem
```

> *Las claves `.pem` son secretos: nunca se commitean, ni se copian a un chat.*

9.6 Generar el pepper de recuperación

El bloque de la sección 9.2 ya lo genera en `infra/.env`. Para crear uno a mano:

```
openssl rand -hex 32
```

Copia el mismo valor en `SECURITY_RESET_PEPPER` del `.env` del servicio de seguridad. Sin pepper el servicio de seguridad se niega a iniciar, del mismo modo que sin las claves RSA. Cambiarlo invalida los códigos de recuperación ya emitidos.

9.7 Decidir el correo de recuperación

Elige una de dos opciones: usar Mailpit en local (recomendado para desarrollo) o usar una cuenta Gmail real con una «app password» de 16 caracteres generada en la cuenta de Google (Google, s. f.). El correo lo envía el servicio de notificaciones: los valores `MAIL_*` van en `infra/.env` y en el `.env` de ese servicio. Para Mailpit usa `MAIL_HOST=localhost`, `MAIL_PORT=587`, `MAIL_USERNAME=dev`, `MAIL_PASSWORD=dev`; los mensajes se leen en `http://localhost:8025` (sección 16).

9.8 Compilar y arrancar

En modo desarrollo, con MySQL, RabbitMQ y el broker arriba (sección 9.3), arranca cada servicio en su propia terminal, en este orden:

```
cd backend/en-monitor-security-service    && ./mvnw spring-boot:run   # :8082
cd backend/en-monitor-device-service      && ./mvnw spring-boot:run   # :8084
cd backend/en-monitor-measurement-service && ./mvnw spring-boot:run   # :8085
```

```
cd backend/en-monitor-home-service        && ./mvnw spring-boot:run   # :8086
cd backend/en-monitor-notification-service && ./mvnw spring-boot:run  # :8087
cd backend/en-monitor-alert-service       && ./mvnw spring-boot:run   # :8088
cd backend/en-monitor-recommendation-service && ./mvnw spring-boot:run # :8089
cd backend/en-monitor-api-gateway         && ./mvnw spring-boot:run   # :8080
# Windows: .\mvnw.cmd spring-boot:run en cada carpeta
```

Para las pruebas, en cada repositorio: `./mvnw clean test` (en Windows, `.\mvnw.cmd clean test`). Cada repositorio incluye el Maven Wrapper, por lo que no hace falta instalar Maven por separado. La API queda en `http://localhost:8080` (el gateway); la documentación OpenAPI de cada servicio en `/v3/api-docs` y Swagger UI en `/swagger-ui.html` de su propio puerto (`http://localhost:8082` y `:8084` a `:8089`) (Spring, s. f.). El estado de salud está en `/actuator/health` de cada servicio y del gateway.

> *Las pruebas no son herméticas: las de persistencia corren contra la base de datos real (Liquibase crea el esquema). Por eso MySQL debe estar sano antes de* `mvnw test`*. El gateway no necesita base de datos.*

10. Instalación del frontend web

Repositorio: `Energy-Monitor-application` (React 19 + Vite 8). Dev server en `http://localhost:5173` (Vite, s. f.).

10.1 Dependencias

Con Node 22 y npm en el PATH, desde la carpeta del repositorio:

```
npm install
```

10.2 Configurar el gateway de desarrollo

La web siempre llama a `/api` en su propio origen y el servidor de Vite lo reenvía al gateway de `http://localhost:8080`; así las cookies HttpOnly de la sesión funcionan sin CORS. Solo hace falta un `.env.local` si cambias algún valor por defecto:

```
cp .env.example .env.local
# Windows PowerShell:
Copy-Item .env.example .env.local
```

```
# VITE_DEV_API_TARGET=http://localhost:8080
VITE_GOOGLE_CLIENT_ID=
```

`VITE_GOOGLE_CLIENT_ID` debe ser el mismo ID de cliente que `GOOGLE_CLIENT_ID` del servicio de seguridad. Si queda vacío, el botón de Google no se muestra.

10.3 Arrancar en desarrollo

```
npm run dev
```

Abre `http://localhost:5173`. Para compilar para producción: `npm run build`; para revisar el código: `npm run lint`; para las pruebas: `npm test` (Vite, s. f.).

10.4 Permiso de CORS en el gateway

El CORS lo responde el API gateway. Su `.env` debe listar el origen de Vite como origen permitido:

```
SECURITY_CORS_ALLOWED_ORIGINS=http://localhost:5173
```

> *En la pila completa el origen es `http://localhost` y se define en `infra/.env`. Allí la web y la API comparten origen a través del nginx del frontend, que reenvía `/api/` al gateway.*

11. Instalación de la aplicación móvil

Repositorio: `Energy-Monitor-Mobile` (Expo SDK 57, React Native 0.86.3, TypeScript, solo Android). La aplicación está integrada con la API, por lo que el gateway y los servicios deben estar en ejecución para iniciar sesión y consultar datos (Expo, s. f.).

11.1 Dependencias

```
npm install
```

11.2 Configurar la URL de la API

Copia el ejemplo y define la dirección del gateway, sin `/api/v1`:

```
cp .env.example .env
# Windows PowerShell:
Copy-Item .env.example .env
```

```
EXPO_PUBLIC_API_BASE_URL=http://10.0.2.2:8080
EXPO_PUBLIC_EAS_PROJECT_ID=<id del proyecto en expo.dev>
```

El valor depende de dónde se ejecute la aplicación:

Dónde corre la app	Valor de EXPO_PUBLIC_API_BASE_URL
Emulador de Android	`http://10.0.2.2:8080` (`localhost` sería el propio emulador)
Teléfono físico	`http://<IP del PC en la red local>:8080` en modo desarrollo, por ejemplo `http://192.168.1.50:8080`, o `http://<IP del PC>` con la pila completa
Navegador (`npm run web`)	`http://localhost:8080`

Expo incorpora las variables `EXPO_PUBLIC_*` al compilar: tras cambiar el valor, reinicia `npx expo start`. CORS no aplica a React Native, por lo que no hay que agregar la app a `SECURITY_CORS_ALLOWED_ORIGINS`. El token de push de Expo (`EXPO_ACCESS_TOKEN`) nunca va en este archivo: es del servicio de notificaciones. Si se usa el inicio de sesión con Google en la app, agrega `EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID` con el ID de cliente **web** y crea en el mismo proyecto de Google un cliente Android con el paquete `com.energymonitor.mobile` y el SHA-1 de la llave que firma el APK.

11.3 Arrancar Expo

```
npm start
# o directamente:
npx expo start
```

El CLI de Expo muestra un código QR y menús interactivos. Elige la plataforma:

- Android físico o emulador: `npm run android` (o presiona `a`).
- Navegador, solo como apoyo en desarrollo: `npm run web` (o presiona `w`).

11.4 Probar en un dispositivo físico

Instala Expo Go desde Google Play, asegúrate de que el equipo y el móvil estén en la misma red, y escanea el QR desde Expo Go. Es la vía recomendable en Windows porque no requiere Android Studio (Expo, s. f.). Para que el teléfono llegue al gateway, permite el puerto 8080 (modo desarrollo) o el 80 (pila completa) en el firewall del equipo.

11.5 Notas de compatibilidad

- Node 22 es el entorno de referencia para el tooling de Expo SDK 57.
- La aplicación móvil no está disponible para iPhone: el manual no cubre iOS ni Xcode.
- La vinculación del módulo desde la app usa Bluetooth nativo (`react-native-ble-plx`): concede el permiso de Bluetooth cuando la app lo pida. Las notificaciones push requieren `EXPO_PUBLIC_EAS_PROJECT_ID` en la app y `EXPO_ACCESS_TOKEN` en el servidor.
- La app guarda la sesión en el almacén seguro del sistema y tiene desactivadas las copias de seguridad de Android: al reinstalarla hay que volver a iniciar sesión.
- La vinculación del módulo desde la app usa Bluetooth nativo (`react-native-ble-plx`): concede el permiso de Bluetooth cuando la app lo pida. Las notificaciones push requieren `EXPO_PUBLIC_EAS_PROJECT_ID` en la app y `EXPO_ACCESS_TOKEN` en el servidor.
- La app guarda la sesión en el almacén seguro del sistema y tiene desactivadas las copias de seguridad de Android: al reinstalarla hay que volver a iniciar sesión.
- Si el QR no abre, comprueba el firewall o inicia con `npx expo start --tunnel`.

12. Broker MQTT

El broker es Eclipse Mosquitto con el complemento `mosquitto-go-auth` y su configuración vive en `energy-monitor-infra/mosquitto/`. No guarda usuarios: por cada conexión y cada publicación pregunta al servicio de dispositivos (`POST /internal/mqtt/auth/user` y `/acl`, con caché de 5 minutos), y cada dispositivo solo puede publicar en sus propios tópicos. Se levanta con la compose de infraestructura (sección 9.3), en la pila completa o en modo desarrollo (Eclipse Foundation, s. f.).

Clientes que acepta el broker: el servicio de dispositivos (`em-device-service`, lee telemetría y estado y es el único que publica comandos), el servicio de mediciones (`em-measurement-service`, solo lee la telemetría para guardar las lecturas) y cada dispositivo ESP32 (`device_code` y `api_key`). Ningún otro servicio se conecta al broker: el servicio de alertas recibe cada lectura por RabbitMQ (`measurement.recorded`).

Pasos:

1. Define en `infra/.env` `DEVICE_MQTT_PUBLIC_HOST` (la IP del PC en la red local; en Linux, `hostname -I`), `DEVICE_MQTT_PUBLIC_PORT=8883`, `MQTT_DEVICE_SERVICE_PASS` y `MQTT_MEASUREMENT_SERVICE_PASS` (sección 9.2); en modo desarrollo, los mismos valores en los `.env` de los servicios de dispositivos y de mediciones (sección 9.4).
2. Genera el certificado TLS de desarrollo con `./prepare.sh` desde `infra/` (llama a `mosquitto/generate-dev-certs.sh`). El script crea una autoridad certificadora de desarrollo y el certificado del broker en `infra/mosquitto/certs/`, válido para las IP del equipo y siempre para `DEVICE_MQTT_PUBLIC_HOST`, y escribe `backend/energy-monitor-backend/firmware/esp32-pzem-monitor/dev_ca.h` para que el firmware confíe en

esa autoridad. Si la IP del equipo cambia, actualiza `DEVICE_MQTT_PUBLIC_HOST` y vuelve a ejecutarlo: la autoridad se reutiliza y los dispositivos no necesitan reprogramarse.

3. En modo desarrollo, el broker llama al servicio de dispositivos que corre en el equipo (por `host-gateway`). En Linux con ufw, permite esa llamada una sola vez, solo desde las redes de Docker:

```
sudo ufw allow from 172.16.0.0/12 to any port 8084 proto tcp comment 'mosquitto -> device'
```

4. Con el servicio de dispositivos en ejecución, revisa el broker desde `infra/`:

```
docker compose logs -f mosquitto
```

Los servicios aparecen conectados como `em-device-service` y `em-measurement-service`. Los dispositivos usan el puerto 8883 con TLS; el 1883 es interno, solo para los servicios, y Docker lo publica únicamente en el bucle local. Para volver a la pila completa, `docker compose up -d` recrea el broker sin `host-gateway`.

> *Si el contenedor se cierra con un error de permisos sobre `server.key`, el contenedor lee el archivo como el usuario 1000: ejecuta `sudo chown 1000 infra/mosquitto/certs/server.key`.*

13. Firmware del dispositivo de medición

El firmware (versión 1.3.8) está en `backend/energy-monitor-backend/firmware/esp32-pzem-monitor/` (repositorio `energy-monitor-backend`) y se compila con arduino-cli o con Arduino IDE 2 (Arduino, s. f.). No hay actualización por aire (OTA): el firmware se carga siempre por USB.

13.1 Cableado

PZEM-004T V3	ESP32	Nota
TX	GPIO16 (RX2)	Se cruzan TX y RX
RX	GPIO17 (TX2)	
VCC	3V3	Si no responde, usar 5 V desde VIN
GND	GND	Común con el PZEM

La comunicación es Modbus RTU a 9600 baudios, 8N1, con dirección `0xF8`. La fuente HLK-PM03 alimenta la placa desde el mismo circuito que se mide.

> **Advertencia de seguridad:** el PZEM-004T no está aislado galvánicamente. No toques el lado conectado a la red eléctrica ni manipules el cableado de la red con el prototipo energizado.

13.2 Herramientas y librerías

```
arduino-cli core update-index
arduino-cli core install esp32:esp32
arduino-cli lib install PZEM004Tv30 PubSubClient ArduinoJson WiFiManager NimBLE-Arduino
```

Librería	Autor	Versión probada
PZEM004Tv30	Jakub Mandula	1.2.1
PubSubClient	Nick O'Leary	2.8
ArduinoJson	Benoit Blanchon	7.x
WiFiManager	tzapu	2.0.17
NimBLE-Arduino	h2zero	2.5.1

No confundas `PZEM004Tv30` con la librería antigua `PZEM-004T` (v1/v2).

13.3 Preparación

```
cd backend/energy-monitor-backend/firmware/esp32-pzem-monitor
cp secrets.h.example secrets.h
```

Define al menos `PORTAL_PASSWORD` (8 caracteres o más) en `secrets.h`: protege el punto de acceso `EnergyMonitor-Setup`. El archivo `dev_ca.h` lo genera `prepare.sh` de `energy-monitor-infra` (sección 12). Ambos archivos están ignorados por Git.

13.4 Compilar y cargar por USB

El esquema de particiones debe ser **Minimal SPIFFS (1.9 MB APP with OTA)**, porque con Bluetooth el binario (~1,4 MB) no cabe en el esquema por defecto.

```
arduino-cli compile --fqbn esp32:esp32:esp32:PartitionScheme=min_spiffs .
arduino-cli upload  --fqbn esp32:esp32:esp32:PartitionScheme=min_spiffs -p /dev/ttyUSB0 .
arduino-cli monitor -p /dev/ttyUSB0 -c baudrate=115200
```

En Arduino IDE: placa **ESP32 Dev Module** y *Partition Scheme* **Minimal SPIFFS**. Cierra cualquier otro monitor serie antes de cargar, o el puerto estará ocupado. En Linux, el usuario necesita acceso al puerto (`sudo usermod -aG dialout $USER` y volver a iniciar sesión).

13.5 Modo de configuración

Un dispositivo nuevo, sin configuración, entra solo en el modo de configuración. Para activarlo en uno ya configurado, mantén pulsado el botón **BOOT** (GPIO0) durante 3 segundos. Sin red conocida durante 5 minutos, el dispositivo ofrece la configuración por Bluetooth por sí solo.

14. Vincular el primer dispositivo

El dispositivo se registra desde la aplicación web o desde la app móvil; no hace falta insertar nada a mano en la base de datos. Requisitos: base de datos, RabbitMQ, broker, gateway y servicios en ejecución, y la web abierta en Chrome o Edge de escritorio, o Chrome en Android, mediante HTTPS o `localhost` (en la app móvil, Bluetooth encendido y su permiso concedido).

1. En la web o la app: hogar → **Dispositivos** → **Vincular dispositivo** (solo el propietario).
2. Mantén pulsado **BOOT** en la ESP32 durante 3 segundos hasta que el LED parpadee (un dispositivo nunca vinculado ya está en modo de configuración tras cargarlo).
3. **Buscar módulo**: el navegador (o la lista de la app) muestra `EnergyMonitor-<código>`; selecciónalo.
4. Elige el electrodoméstico, el nombre y la habitación.
5. Elige la red Wi-Fi del hogar de la lista (son las redes que ve la ESP32) y escribe su contraseña.
6. La web o la app registra el dispositivo en el servicio de dispositivos, que genera su `api_key`, y envía al dispositivo por Bluetooth la red, la contraseña, el servidor (`DEVICE_MQTT_PUBLIC_HOST`), el identificador y la clave.
7. Al conectarse al broker, el dispositivo sale solo del modo de configuración.

Alternativa sin Bluetooth (Safari, Firefox u otros navegadores): conéctate a la red Wi-Fi `EnergyMonitor-Setup` con la clave `PORTAL_PASSWORD`, abre `http://192.168.4.1` → **Configure WiFi** y completa la red, el servidor

MQTT (la IP del PC o el dominio del broker), el puerto 8883, el identificador, el código y la clave del dispositivo. Esta vía exige que el dispositivo ya exista en el servicio de dispositivos con su clave.

> *En Linux, Chrome trae Web Bluetooth desactivado: habilita* `chrome://flags/#enable-experimental-web-platform-features` *y reinicia el navegador. Web Bluetooth solo funciona en* `https://` *o* `http://localhost`.

Salida esperada en el monitor serie:

```
=== EnergyMonitor ESP32 + PZEM-004T V3 fw=1.3.8 ===
[PZEM] UART2 ready at 9600 baud, addr=0xf8
[WiFi] connected to 'MiHogar', IP=192.168.1.37
[MQTT] connected as esp32-DEV0000042
[MQTT] telemetry published
```

15. Producción

Esta sección resume el despliegue en un servidor con Docker Compose desde `energy-monitor-infra` (`docker-compose.prod.yml` y `.env.prod`): MySQL, RabbitMQ, el broker, el API gateway, los siete servicios y el frontend. El procedimiento completo, con verificación, reversión y despliegue continuo, está en el manual de implementación y despliegue. El `mosquitto.conf` es el mismo archivo que en desarrollo (Let's Encrypt, s. f.).

1. **Servidor y dominio.** Un servidor con Docker, Nginx y Certbot, y registros DNS del dominio de la web, de `www` y del broker (por ejemplo `mqtt.ejemplo.com`) apuntando a él.
2. **Código.** Los repositorios clonados uno al lado del otro en `/opt/energy-monitor` (`infra`, el gateway y los siete servicios en `backend/`, el backend retirado solo por el firmware y `frontend/energy-monitor-web`), como en la sección 9.1, en la rama `main`.
3. **Certificado del broker.** Obtén un certificado de Let's Encrypt para el dominio del broker (`certbot certonly --nginx -d mqtt.ejemplo.com`); el hook `deploy/renew-mqtt-cert.sh` copia `fullchain.pem` como `server.crt` y `privkey.pem` como `server.key` en la carpeta `MQTT_CERT_DIR` (legible por el usuario 1000) y reinicia `mosquitto` en cada renovación. El firmware no cambia: confía en las raíces de Let's Encrypt.
4. **Configuración.** `cp .env.prod.example .env.prod` en `infra/` y complétalo: contraseñas de MySQL, pepper, `RABBITMQ_PASSWORD`, `INTERNAL_API_TOKEN`, `MQTT_DEVICE_SERVICE_PASS`, `MQTT_MEASUREMENT_SERVICE_PASS`, la carpeta del par de claves JWT, `DEVICE_MQTT_PUBLIC_HOST` con el dominio del broker, `https://<dominio>` como origen y URL pública, correo, Google, VAPID y Expo. La clave privada JWT debe ser legible por `BACKEND_UID` (10001 por defecto), el usuario con el que corre el servicio de seguridad; `public.pem`, por todos. `chmod 600 .env.prod`.
5. **Arranque.**

```
cd infra
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build
```

6. **Imágenes.** El `Dockerfile` de cada servicio Java compila con Maven y ejecuta con un JRE 21 como usuario sin privilegios (UID 10001). Ningún servicio publica puertos: solo se publican la web en `127.0.0.1:8081` (por defecto), MQTT con TLS en 8883 y la consola de RabbitMQ en `127.0.0.1:15672`.
7. **Cortafuegos.** Abre los puertos 22, 80, 443 (proxy HTTPS) y 8883 (MQTT con TLS). Mantén cerrados el 1883, el 3306, el 8081 y el 15672.
8. **Proxy inverso HTTPS** (Nginx con `deploy/nginx-energy-monitor.conf`) delante de `127.0.0.1:8081` para la web y la API. El nginx del frontend solo pasa `/api/` al gateway, y el gateway rechaza `/internal/**`. La web debe servirse por HTTPS, porque Web Bluetooth y el push lo exigen.
9. **Dirección real del cliente.** El sitio de Nginx reemplaza `X-Forwarded-For` por la IP real, y el gateway y el servicio de seguridad solo lo aceptan si la conexión viene de la red de Docker; así el límite de intentos no se comparte entre todos los usuarios.

10. **Primer administrador y respaldos.** Registra la cuenta en la web y ejecuta `./make-admin.sh --prod correo@ejemplo.com`; programa `deploy/backup-mysql.sh` en el cron de root.

Las claves VAPID se generan una sola vez por entorno (cambiarlas invalida todas las suscripciones push); los comandos están en el README de `energy-monitor-infra`. Varias instancias de los servicios de dispositivos o de mediciones necesitan valores distintos de `MQTT_CLIENT_ID`. Un servidor que todavía corre `energy-monitor-backend` se migra con el procedimiento «Retiro de energy-monitor-backend» del README de `energy-monitor-infra` (respaldo, vaciar colas de alertas, comprobar `backend_outbox`, `deploy.sh infra` y `deploy.sh gateway`, borrar las colas `energy-monitor-backend.*`). Cuando se despliega un servicio que consume eventos nuevos, debe desplegarse **antes** que el que los produce: un outbox se detiene ante un mensaje que ninguna cola recibe.

16. Correo local con Mailpit (opcional)

Mailpit es un servidor SMTP de desarrollo: acepta cualquier mensaje y los muestra en una bandeja web sin enviar nada por internet real. Se levanta con Docker (Mailpit, s. f.):

```
docker run -d --name mailpit -p 587:1025 -p 8025:8025 \
  -e MP_SMTP_AUTH_ACCEPT_ANY=1 -e MP_SMTP_AUTH_ALLOW_INSECURE=1 axllent/mailpit
```

Usa el puerto 587 del equipo (en lugar del 1025 por defecto de Mailpit) para que al alternar entre Mailpit y Gmail solo cambie `MAIL_HOST` y las credenciales. En el `.env` del servicio de notificaciones (modo desarrollo):

```
MAIL_HOST=localhost
MAIL_PORT=587
MAIL_USERNAME=dev
MAIL_PASSWORD=dev
MAIL_FROM=Energy Monitor <no-reply@localhost>
```

Bandeja de entrada del desarrollo: `http://localhost:8025`. Cuando uses Gmail, el código de recuperación llega al buzón real asociado a la cuenta con la «app password».

17. Verificación de la instalación

Ejecuta esta lista antes de dar por terminada la instalación. Cada comando debe producir el resultado indicado:

Componente	Comprobación	Resultado esperado
Docker	`docker compose ps` en `infra/`	`energy-monitor-mysql` y `energy-monitor-rabbitmq` en `healthy`; en la pila completa, también gateway, security, device, measurement, home, notification, alert y recommendation `(healthy)`.
Web y API	`curl -s -o /dev/null -w '%{http_code}\n' http://localhost/` (pila completa)	`200`.
Gateway	`GET http://localhost/api/v1/homes` (pila completa) o `http://localhost:8080/api/v1/homes` (modo desarrollo), sin sesión	`401`: lo responde el gateway.
Rutas por servicio	`POST /api/v1/auth/login` con `{}`; `GET /api/v1/appliance-types`, `/consumption-levels`, `/home-types`, `/alerts?homeId=x`, `/recommendations?homeId=x`, `/notifications/preferences`; `GET /api/v1/otra-ruta`	`400` (llegó al servicio de seguridad); `401` en las demás (rutas de cada servicio); `404` en la ruta que nadie atiende.
Servicios	`http://localhost:8080/actuator/health`, `:8082` y `:8084` a `:8089` (modo desarrollo)	Cada uno responde `UP`.

Componente	Comprobación	Resultado esperado
Swagger	`http://localhost:8082/swagger-ui.html` (y `:8084` a `:8089`) en modo desarrollo	Swagger UI lista la documentación de cada servicio.
OpenAPI	`GET http://localhost:8086/v3/api-docs` (u otro puerto de servicio)	JSON con la especificación de OpenAPI.
Broker	`docker compose logs mosquitto | grep "New client connected"` en `infra/`	Conectados `em-device-service` y `em-measurement-service`; puerto 8883 publicado.
RabbitMQ	`http://localhost:15672` (`RABBITMQ_USERNAME` / `RABBITMQ_PASSWORD`)	La consola abre y las colas `.dlq` están vacías.
Dispositivo	Monitor serie del ESP32	`[MQTT] connected as esp32-<identificador>` y `[MQTT] telemetry published`.
Primera medición	Panel del hogar en la web	La primera medición del dispositivo aparece en la web en aproximadamente un minuto.
Web (desarrollo)	`npm run dev` en Energy-Monitor-application	Vite en `http://localhost:5173` muestra la pantalla de acceso.
Móvil	`npm start` en Energy-Monitor-Mobile	Expo muestra el QR y, al abrirlo con Expo Go, la app permite iniciar sesión contra la API.
Correo	`MAIL_HOST=localhost` + Mailpit en :587	Los códigos de recuperación aparecen en `http://localhost:8025`.
Administrador	`./make-admin.sh correo@ejemplo.com` en `infra/` y volver a iniciar sesión en la web	Aparece **Administración** en el menú lateral y la pantalla `/admin/users`.
Administrador	`./make-admin.sh correo@ejemplo.com` en `infra/` y volver a iniciar sesión en la web	Aparece **Administración** en el menú lateral y la pantalla `/admin/users`.

Para una comprobación de punta a punta: registra un usuario desde la web, verifica su correo (Mailpit o Gmail), inicia sesión, crea un hogar, vincula el dispositivo (sección 14) y confirma que su primera medición aparece (Cortés Parra y García Artunduaga, 2026). Con la pila completa arriba, los scripts de `infra/e2e/` (`device-flow.sh`, `measurement-flow.sh`, `activity-flow.sh`, `alert-flow.sh`, `recommendation-flow.sh` y `account-deletion-flow.sh`) recorren automáticamente lo que cruza los servicios (rutas del gateway, vinculación, broker, lecturas, alertas, recomendaciones, actividad y borrado de cuenta) y terminan con el número de comprobaciones aprobadas; usan `jq` y `openssl`.

18. Resumen de comandos de instalación

Paso	Infraestructura (energy-monitor-infra)	Servicios Java (gateway y los siete servicios)	Web (Energy-Monitor-application)	Móvil (Energy-Monitor-Mobile)
Variables	`cp .env.example .env` y secretos (9.2); primer administrador con `./make-admin.sh`	`cp .env.example .env` y `application-local.yaml` (9.4, modo desarrollo)	`cp .env.example .env.local` (opcional)	`cp .env.example .env`
Deps.	—	(wrapper Maven incluido)	`npm install`	`npm install`
Claves/certificados	`./prepare.sh`	Rutas de las claves en `.env` (9.5)	—	—
Infraestructura	Pila completa: `docker compose up -d --build`; desarrollo: `docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d mysql rabbitmq mosquitto`	—	—	—

Paso	Infraestructura (energy-monitor-infra)	Servicios Java (gateway y los siete servicios)	Web (Energy-Monitor-application)	Móvil (Energy-Monitor-Mobile)
Pruebas	`./e2e/*.sh` (pila completa)	`./mvnw test` (`.\mvnw.cmd test` en Windows)	`npm test`	`npm run lint`
Ejecutar	—	`./mvnw spring-boot:run` en cada servicio (9.8)	`npm run dev`	`npm start`
URL típica	`http://localhost` (pila completa)	`http://localhost:8080` (gateway)	`http://localhost:5173`	QR / localhost:8081 (Expo)

Broker: `./prepare.sh` en `infra/` genera sus certificados y la compose lo levanta. Firmware (`backend/energy-monitor-backend/firmware/esp32-pzem-monitor`): `arduino-cli compile` y `arduino-cli upload` (sección 13).

19. Solución de problemas habituales

Síntoma	Causa probable	Acción correctiva
`required variable X is missing a value` al levantar la pila	`X` está vacía en `infra/.env` o falta su línea.	Rellenarla (sección 9.2) y repetir `docker compose config -q` hasta que no nombre ninguna.
El servicio de seguridad no arranca y nombra un path de clave o el pepper	Faltan `SECURITY_JWT_PRIVATE_KEY_PATH`, `SECURITY_JWT_PUBLIC_KEY_PATH` o `SECURITY_RESET_PEPPER`.	Definirlas en su `.env` (rutas absolutas) y reintentar.
`Access denied for user 'energy_app'` en un servicio o al correr tests	Su `SPRING_DATASOURCE_PASSWORD` o su `application-local.yaml` no coinciden con `MYSQL_PASSWORD` de `infra/.env`.	Copiar el valor de `MYSQL_PASSWORD` (sección 9.4).
Un servicio no arranca por un valor del `.env`	Un valor con comillas o con `$` (el archivo se lee como `.properties`).	Escribir los valores sin comillas y sin `$`.
Un servicio queda `unhealthy` con RabbitMQ arriba	`RABBITMQ_USERNAME` / `RABBITMQ_PASSWORD` no coinciden con el usuario que RabbitMQ creó la primera vez que arrancó su volumen.	Cambiar la clave dentro de RabbitMQ (`rabbitmqctl change_password`, ver README de infra) o borrar el volumen `rabbitmq-data` (se pierden los mensajes pendientes).
La API responde `503`	El gateway no alcanza al servicio de esa ruta.	Revisar `docker compose ps` y `docker compose logs security device measurement home notification alert recommendation` (o la terminal del servicio en modo desarrollo).
La API responde `404` en una ruta de `/api/v1`	Ningún servicio atiende esa ruta (el backend retirado ya no recibe «el resto»).	Revisar la ruta o desplegar la versión del gateway que la enruta.
El navegador bloquea las llamadas desde la web	Falta `SECURITY_CORS_ALLOWED_ORIGINS` con el origen de Vite en el gateway.	Agregarlo en el `.env` del gateway (sección 10.4).
El frontend repite `401` en `/api/v1/homes`	Es lo esperado sin sesión.	Iniciar sesión (registro o Google).
No llegan los códigos de verificación o recuperación	Correo mal configurado en el servicio de notificaciones; el error queda en la tabla `notification`.	Revisar `MAIL_*` del servicio de notificaciones, la cola `notification-service.notification.mail` y su `.dlq`, usar Mailpit y leer la bandeja en `:8025` (sección 16).

Síntoma	Causa probable	Acción correctiva
Un evento no llega (alertas, actividad, borrado de hogar)	El outbox del productor está detenido porque la cola del consumidor no existe.	Consultar `last_error` de la tabla `*_outbox`, desplegar el consumidor y esperar a que el relay avance.
Llegan lecturas pero no se crean alertas	El servicio de alertas está detenido o con `ALERT_CONSUMERS_ENABLED=false`.	Iniciar `alert`, quitar el `false` de `.env` y revisar la cola `alert-service.measurement.recorded`.
Mailpit/Docker no arranca	Docker Desktop detenido o puerto ocupado.	Iniciar Docker Desktop; cambiar puerto con `-p 1587:1025` si 587 está en uso.
El dispositivo no conecta (`[MQTT] connection failed, rc=-2` o `X509 - Certificate verification failed`)	`DEVICE_MQTT_PUBLIC_HOST` apunta a `localhost` o a una IP equivocada, el certificado no es válido para la IP actual o la red lo bloquea.	Usar la IP del PC en la red local, volver a ejecutar `./prepare.sh`, `docker compose up -d --force-recreate mosquitto` y revisar el firewall.
El dispositivo muestra `rc=4` o `rc=5`	El código o la clave no coinciden con los del servicio de dispositivos, o ese servicio está detenido.	Iniciar el servicio de dispositivos y volver a vincular el dispositivo.
Un servicio es rechazado por el broker (`rc=5`)	`MQTT_MEASUREMENT_SERVICE_USER/PASS` o `MQTT_DEVICE_SERVICE_USER/PASS` no son los mismos en quien se conecta y en el servicio de dispositivos.	Igualar los valores en `infra/.env` y en los `.env` de los dos servicios.
El broker registra `context deadline exceeded` al llamar a `device:8084` (modo desarrollo)	El servicio de dispositivos del equipo está detenido o falta la regla de ufw.	Iniciar el servicio de dispositivos y aplicar la regla de la sección 12 (con el puerto 8084).
Los servicios tardan unos 2 minutos en el primer arranque	Liquibase ejecuta sus changesets sobre la base nueva.	Es normal; esperar a `(healthy)` en `docker compose ps`.
La web no encuentra el dispositivo por Bluetooth	El dispositivo no está en modo de configuración, el navegador no es compatible o la página no es segura.	Mantener BOOT 3 s hasta que parpadee el LED, usar Chrome o Edge y abrir la web en `localhost` o por HTTPS; en Linux habilitar el flag de la sección 14.
Falla `wifi_failed:auth` al vincular	Contraseña de Wi-Fi incorrecta.	Volver a vincular con la contraseña correcta.
El push del navegador no llega	Faltan las claves VAPID o el usuario no concedió el permiso de notificaciones; el navegador exige HTTPS o `localhost`.	Definir `NOTIFICATION_VAPID_PUBLIC_KEY` y `NOTIFICATION_VAPID_PRIVATE_KEY`, reiniciar el servicio de notificaciones y activar las notificaciones en el navegador.
Google responde `origin_mismatch` o no aparece el botón	Falta el origen en *Authorized JavaScript origins* del cliente OAuth, o `GOOGLE_CLIENT_ID` / `VITE_GOOGLE_CLIENT_ID` están vacíos.	Registrar `http://localhost` y `http://localhost:5173` en Google Cloud Console, configurar las variables y reconstruir el frontend o reiniciar Vite.
Falla la carga del firmware: puerto ocupado	Otro programa usa el puerto serie.	Cerrar el monitor serie de Arduino IDE o cualquier otro programa que use `/dev/ttyUSB0`.
«Sketch too big» al compilar	El esquema de particiones no es Minimal SPIFFS.	Compilar con `PartitionScheme=min_spiffs` (sección 13.4).
«~» no se expande en la ruta de claves	Java no expande el símbolo en las rutas de las claves.	Usar la ruta absoluta completa (`C:\Users\tu-usuario\.energy-monitor\keys\`).
La app móvil no llega a la API	`EXPO_PUBLIC_API_BASE_URL` apunta a `localhost`, que en el emulador o en el teléfono es el propio equipo.	Usar `10.0.2.2` en el emulador o la IP del PC en el teléfono y reiniciar Expo (sección 11.2).
Expo no conecta con Expo Go	Distinta red, firewall, o QR inválido.	Usar `npx expo start --tunnel`; verificar scripts de red corporativa.

Síntoma	Causa probable	Acción correctiva
`npm install` falla en móvil/web	Versión de Node distinta a la del software de referencia.	Actualizar a Node 22 LTS (mín. 20.19) y reinstalar.

20. Referencias (formato APA, 7.ª edición)

Las citas en el texto y la lista de referencias siguen el estilo de la séptima edición del manual de publicaciones de la APA (American Psychological Association [APA], 2020).

American Psychological Association. (2020). Publication manual of the American Psychological Association (7.ª ed.). https://www.apa.org/

Arduino. (s. f.). Arduino CLI. https://arduino.github.io/arduino-cli/

Cortés Parra, J. E. y García Artunduaga, M. A. (2026). *Especificación de requisitos de software de EnergyMonitor* (versión 2.0). SENA – CIES Regional Huila.

Docker Inc. (s. f.). Docker documentation. https://docs.docker.com/

Eclipse Foundation. (s. f.). Eclipse Mosquitto. https://mosquitto.org/

Expo. (s. f.). Expo documentation. https://docs.expo.dev/

Git. (s. f.). Git for Windows. https://git-scm.com/download/win

Google. (s. f.). Cómo crear una contraseña de aplicación. https://myaccount.google.com/apppasswords

Let's Encrypt. (s. f.). Documentation. https://letsencrypt.org/docs/

Mailpit. (s. f.). Mailpit: Web and API based SMTP testing. https://mailpit.axllent.org/

RabbitMQ. (s. f.). RabbitMQ documentation. https://www.rabbitmq.com/docs

MySQL. (2026). MySQL 8.4 reference manual. https://dev.mysql.com/doc/refman/8.4/en/

Node.js Foundation. (s. f.). Node.js. https://nodejs.org/

OpenSSL. (s. f.). OpenSSL cryptography and SSL/TLS toolkit. https://www.openssl.org/

Oracle. (s. f.). Java SE downloads. https://www.oracle.com/java/technologies/downloads/

Spring. (s. f.). Spring Boot reference documentation. https://docs.spring.io/spring-boot/index.html

Vite. (s. f.). Vite guide. https://vite.dev/guide/